CSC 436 Assignment

Database Views & Normalization

Overview

In this assignment, you will enhance your group's database application by implementing SQL views for controlled data access and applying normalization techniques to achieve Third Normal Form (3NF). You will create views to restrict access to specific data and write SQL commands for managing privileges. Additionally, you will analyze your database for normalization violations, make necessary modifications to improve data integrity, and document your work with SQL queries, screenshots, and a detailed explanation of your normalization process.

Part I: SQL Views & Authorization

In this part of the assignment, you will create views in MySQL to control data access and enhance security. Views allow you to present a subset of your data to specific users without granting them full access to the underlying tables. This is particularly useful for protecting sensitive information while still providing necessary data access to different types of users.

Since the MySQL version in your cPanel does not support role-based privileges or granting user-specific permissions through phpMyAdmin, you will also write (but not execute) SQL commands that would typically be used to manage access.

A. Setup

Before creating views, complete the following:

- ☐ Complete the Logging Into cPanel section to access your cPanel account.
- ☐ Since you will be creating views, you need to verify that your database user has the necessary privileges. Complete the Updating A User's Privileges section and ensure your database user has been granted **All Privileges** in phpMyAdmin.
- ☐ Complete the Accessing The Database section to access the database you created.
- ☐ Review the SQL Statements section to familiarize with common SQL statements to effectively communicate with your database.
- ☐ Review the Querying The Database section to write queries and ask questions about the data.

B. Creating Views

You will create at least two views that restrict access to certain data. Consider the following:

- A **limited-access view** that hides sensitive information (i.e., removing customer payment details).
- A **role-based view** that only displays records relevant to a specific user type (i.e., showing only active rental transactions for employees).

After creating your views, they will appear under the **Views** section at the bottom of your database.

C. Simulating Role-Based Access (Written SQL Only, No Execution)

Since the MySQL version in cPanel does not support granting privileges or creating roles, you will write (but not execute) the SQL commands that would typically be used to:

- Grant privileges on a view to a specific user.
- Create a role and assign users to it.

Even though these commands will not execute in your current setup, understanding them is essential for real-world database management.

Part II: Database Normalization

In this part of the assignment, you will analyze and refine your group's database application design to ensure compliance with third normal form (3NF) by identifying and addressing normalization issues.

A. Database Normalization

Normalization helps eliminate redundancy, inconsistencies, and update anomalies by organizing data efficiently. You will review your current database design and apply normalization techniques as needed.

Familiarize yourself with the following rules to analyze your database for normalization violations:

The database follows 1NF, 2NF, and 3NF. Core entity and subtype tables define primary keys and store atomic values. The relationship tables Contains and PlayingIn do not explicitly define primary keys, which could allow duplicate records. Adding composite primary keys would improve data integrity.

Identify non-conforming elements in your existing database schema. Modify tables by restructuring attributes and relationships as needed to ensure that your final design meets 3NF requirements while maintaining functionality. This may involve breaking down tables, creating new tables, and redefining relationships to eliminate redundancy while ensuring data integrity.

Part III: Documentation

You will compile and explain your SQL queries, screenshots, written privilege management commands, and normalization analysis.

1. Document the exact SQL queries you used to define each view. Include a brief explanation of each view's purpose and expected results.

```
CREATE VIEW dm_inventory_view(OwnerName, ItemName, ItemType, ItemDescription, ItemNotes, QuantityOwned) AS
    SELECT Characters.name, Props.name, Props.itemType, Props.description, Props.gmNotes, Props.quantity
    FROM Characters, Props
    WHERE Props.owner = Characters.ID
    ORDER BY Characters.name;
```

This view can be used by a world's game master to view the inventory of each character in their world. It can be useful to check what items the party currently has, along with what loot the party could get from NPCs. Since each tuple in *Props* in the database has an owner, all 5 props are expected to appear in the view along with the name of whoever owns that prop.

```
CREATE VIEW player_NPC_view(Name, Race, Description, Notes, Location) AS
    SELECT Characters.name, Characters.race, Characters.description, Characters.partyNotes, Locations.name
    FROM Characters, Locations
    WHERE Characters.isAt = Locations.ID AND Characters.ID NOT IN (SELECT ID FROM Players)
    ORDER BY Locations.name;
```

On the other hand, this view would be used by the party to see all NPC in the world. This view does not include the *gmNotes* of each character included because those are intended to hold secret information only the game master knows. Also, because the party knows their own characters, any player characters are excluded from the list. Therefore, the 7 characters that are not players should appear in the view, sorted by the location they are in.

2. Take screenshots showing the data displayed under each view. You can use the **Browse** tab and take a screenshot of the data that is displayed under the **Views** section. This will demonstrate that your views were successfully created.

Data of dm_inventory_view:

←T→	OwnerName	ItemName	ItemType	ItemDescription	ItemNotes	QuantityOwned
Edit Copy Delete	Commodore Stephahk	Fort Manifest	Document	A list of all recruits at Hranic Fort	Contains evidence against Stephahk	1
Edit Copy Delete	Horacio Garzon	Grape Wine	Consumable	A fine bottle of Siwaniluan wine	Horacio made this himself	5
Edit Copy Delete	Leon Septar	Desert Cloak	Armor	A cloak that blends into sand	Useful for desert travel	1
Edit Copy Delete	Renly Gokel	Alchemy Kit	Tool	A set of tools for brewing potions	Belongs to Renly	1
Edit Copy Delete	Wilhelm Riquet	Venom Spike	Weapon	A spike harvested from a Kua Lorantene warrior	Can be used as a poison weapon	3

Data of player_NPC_view:

			Name	Race	Description	Notes	Location
Edit	Copy	Delete	Aliyra Maastehr	ghord	Apothecary from Ghordeiol		Ghordeiol
Edit	Copy	Delete	Xiarkydoth	spider	Spider in Myrantahl Forests		Ghordeiol
Edit	Copy	Delete	Commodore Stephahk	human	Recruitment officer at Fort Hranic	Weirdly chill guy	Hranic Fort
Edit	Copy	Delete	Chef Mya	human	Chef at Fort Hranic	Falls in love with Wilhelm	Hranic Fort
Edit	Copy	Delete	Olver Thumbless	human	Recruit who has no thumbs	His name is now 'Nubs'	Hranic Fort
Edit	Copy	Delete	Renly Gokel	human	Recruit specializing in alchemy	He is an evil terrorist	Hranic Fort
Edit	Copy	Delete	Jovi	Ka'Tavin	Animal sold in Siwanilua	Bought by Wilhelm	Siwanilua

3. Include the SQL statements you would have executed to grant privileges on a view and create a role. Document each query's intended purpose, expected results, and actual SQL command.

GRANT SELECT

ON dm_inventory_view

TO Logan

This **grant privileges** authorization ensures that the gamemaster user *Logan* has access to select entities within the *dm_inventory_view* view. While the gamemaster would have privileges to insert, update, or delete items as well, it is best practice to limit that access to the underlying relations, such as *Item, NPC, or Location*. This privilege to the *dm_inventory_view* allows for easy queries that can be postulated by the game master to find information and notes that can only be accessed by themselves. This would result in the user being able to select rows within their view to find specific items or personal inventories.

CREATE ROLE player;

GRANT SELECT

ON player_NPC_view TO player;

GRANT player TO Steve;

GRANT player TO George;

GRANT player TO Henry;

The above SQL commands are constructed to create a role. A role helps categorize users in order to easily grant or revoke permissions to a wide variety of users at once. The above query creates the role *player* which is granted access to *player_NPC_view*. We then categorize the users *Steve, George,* and *Henry* within the *player* role we have created. This will give each of these users SELECT access to player_NPC_view, but not access to the secret notes only for the game master. This is especially useful when another player joins the game. Instead of rewriting a bunch of permissions, we just need to add the new player to the role *player*.

4. Identify and explain each non-conforming element in your database schema. Describe how it violates normalization principles and why it needs to be addressed.

Overall, the database largely follows normalization principles. Core entity tables (Characters, Locations, Props, Users) and subtype tables (Players, NPCs, Creatures) define primary keys and store atomic values, so they comply with 1NF, 2NF, and most 3NF requirements. The remaining no -conforming elements are the relationship tables Contains and PlayingIn, which do not explicitly declare primary keys. Without primary keys, duplicate relationship records could be inserted, which weakens row level uniqueness. Adding composite primary keys to these tables would strengthen 1NF compliance and improve data integrity.

5. Provide a detailed explanation of changes made during the normalization process. Justify each modification, explaining how it improves database efficiency, eliminates redundancy and inconsistencies, and ensures 3NF compliance.

Our big change made during this process was the addition of primary keys for the Contains, PlayingIn,NPCs,Creatures, and Players relationships, which strengthened 1NF because all rows could be uniquely identified. The reason for adding these keys was to add redundancy so duplicate relationships would not cause incorrect query results. Adding these keys strengthened our relationships between entities and ensured uniqueness between each relationship. The efficiency of our database was also improved by this change because it will make expanding relationships easier in the future. Overall that is how we improved our database by making it 3NF compliant.

Submission

When you're finished, complete the following steps to submit your work:

- ☐ Export your document with responses as a **PDF file AND save** it **inside** your **documentation** folder. Refer to the following for documentation on how to do this:
  - Google Docs (*File → Download → PDF Document*)
  - Microsoft Word (*File → Save As / Export → PDF*)
  - Pages (*File → Export To → PDF*)
- ☐ Inside your repo, export your updated database as an **SQL file** into the **db** folder. Follow the steps in the Exporting A Database section to export your database into a single SQL file.
- ☐ All group members should then be able to pull this SQL file from the repo and upload it to their own cPanel accounts (see Importing A Database section).
- ☐ Upload all your changes to GitHub.
  - ☐ **If you're using GitHub Desktop (GUI)**, complete the Uploading Changes (GitHub Desktop) section to upload your changes from your local device to GitHub.
  - ☐ **If you're using Git (CLI)**, complete the Uploading Changes (GitHub CLI) section to upload your changes from your local device to GitHub.

ONE group member must paste the URL of your GitHub repository in the provided textbox in Brightspace. Click the blue *Submit* button to successfully submit your work for this assignment.

Grading Rubric

You can refer to the **Database Views & Normalization grading rubric** given in Brightspace for this assignment to find details on how your submission will be graded.